Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

Heart Disease Risk Prediction:

An End-to-End MLOps Pipeline from Data to Monitored Kubernetes API

Student name	A Swathi
BITS ID	2025AE05795
Course	AIMLCZG523 — Machine Learning Operations
Code repository	https://github.com/SwathiVeerapalli/heart-disease-mlops
Pipeline runs evidenced	GitHub Actions ml-pipeline runs #1, #3 and #4 — all 7 jobs passed
Submission date	October 2026

1. Introduction and Summary

The objective of this assignment is to build a machine learning classifier that predicts the presence of heart disease from routine patient measurements, and to take it all the way to a cloud-ready, monitored API using MLOps practices: reproducible data processing, experiment tracking, automated testing, CI/CD, containerisation, Kubernetes deployment and observability.

The solution is organised as a single GitHub repository. Every step, from downloading the UCI data to deploying the API on Kubernetes and capturing monitoring dashboards, runs automatically in a seven-stage GitHub Actions pipeline on every push to `main`. The evidence in this report (metrics, screenshots and logs) was produced by that pipeline and committed back to the repository by the pipeline itself, so each result can be traced to a specific workflow run.

Table 1. *Summary of results*

Area	Outcome
Data	UCI Cleveland heart disease data, 303 patients, 13 features; 6 missing values imputed inside the model pipeline
Models	Logistic Regression and Random Forest, tuned with GridSearchCV (16 and 18 configurations) and 5-fold stratified CV
Selected model	Logistic Regression (C = 0.1, L2): CV ROC-AUC 0.912 ± 0.017; hold-out ROC-AUC 0.962, accuracy 0.885, recall 0.929
Tracking	MLflow: parameters, CV/test metrics, tuning grids, plots and signed models logged for every run; champion tagged
CI/CD	lint → test (20 tests) → train → docker → deploy-k8s ‖ monitoring → publish; run #1 succeeded in 8 min 34 s
Serving	FastAPI `/predict` in a non-root Docker image, pushed to GHCR; 2 replicas on Kubernetes behind NGINX Ingress
Monitoring	Structured JSON request logs; Prometheus metrics; provisioned 6-panel Grafana dashboard

2. System Architecture

Figure 1 shows the three layers of the system. The **ML pipeline** (Python package `heart/`) acquires and cleans data, produces EDA figures, trains and tunes candidate models and logs everything to MLflow. The **CI/CD layer** orchestrates those steps and adds quality gates and deployment. The **runtime layer** serves the exported model as a containerised FastAPI service on Kubernetes, with Prometheus scraping the service and Grafana visualising it.

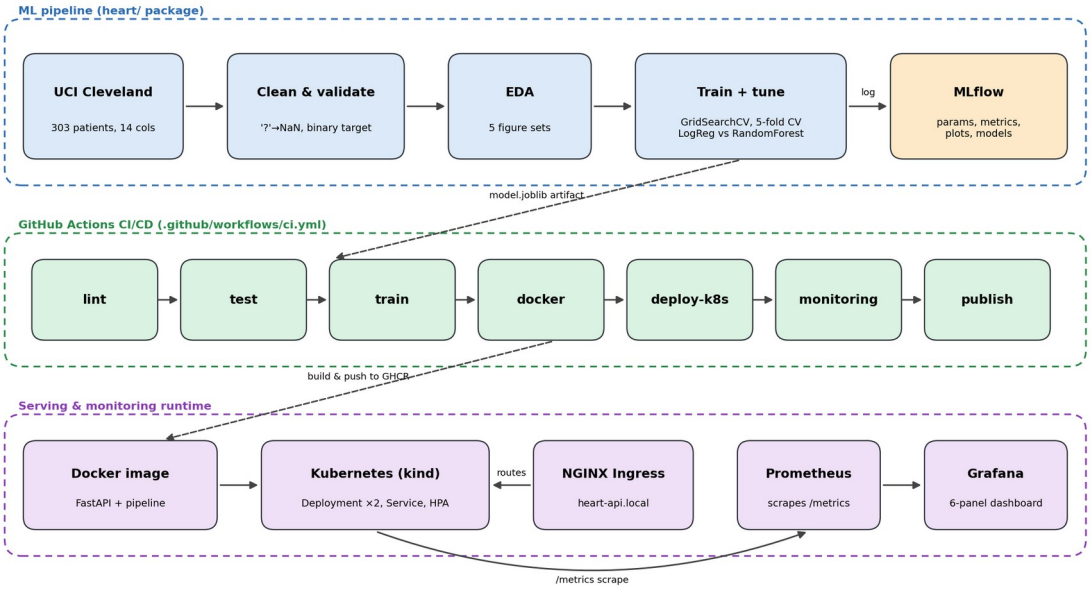


Figure 1. *End-to-end architecture: ML pipeline, GitHub Actions CI/CD and serving/monitoring runtime*

A key design decision is that preprocessing and the classifier are saved together as one scikit-learn `Pipeline`. The API therefore receives raw clinical values and applies exactly the same imputation, scaling and encoding that was used in training, removing a common source of training/serving skew.

Table 2. *Repository layout*

Path	Contents
`heart/`	download.py, data.py, features.py, eda.py, train.py, predict.py (inference), config.py
`app/main.py`	FastAPI service: /predict, /health, /model-info, /metrics, JSON request logging
`test/`	Pytest suites for data, features, model training and the API (20 tests)
`.github/workflows/ci.yml`	Seven-job CI/CD pipeline
`Dockerfile`, `requirements*.txt`	Serving image and pinned training / serving / dev dependencies
`k8s/`	Namespace, Deployment, LoadBalancer Service, Ingress, HPA; `k8s/kind/` cluster config
`monitoring/`	docker-compose stack, Prometheus config, Grafana provisioning + dashboard, load generator
`data/processed/`, `reports/`, `screenshots/`	Cleaned dataset, EDA and model figures, and run evidence (committed by CI)

3. Setup and Run Instructions

3.1 Prerequisites

Python 3.11, Git and Docker are required. For the Kubernetes steps either Minikube or kind with kubectl is needed. All Python dependencies are pinned in `requirements.txt` (training and EDA), `requirements-serve.txt` (API runtime) and `requirements-dev.txt` (tests and linting).

3.2 Train and evaluate from a clean checkout

```
git clone https://github.com/SwathiVeerapalli/heart-disease-mlops.git && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

```
pip install -r requirements-dev.txt

python -m heart.download      # download UCI file, clean, validate ->
data/processed/heart_clean.csv
python -m heart.eda           # figures -> reports/figures/
python -m heart.train         # tuning + CV, MLflow runs in ./mlruns, model ->
models/model.joblib
mlflow ui --backend-store-uri ./mlruns     # http://localhost:5000
pytest -v                     # 20 unit tests
ruff check . && ruff format --check .
```

3.3 Serve the model

```
uvicorn app.main:app --port 8000                    # Swagger UI at
http://localhost:8000/docs
docker build -t heart-api:1.0.0 . && docker run --rm -p 8000:8000 heart-api:1.0.0
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d
@sample_request.json
```

3.4 Deploy to Kubernetes

```
# Minikube (LoadBalancer service)
minikube start && minikube image load heart-api:1.0.0
kubectl apply -f k8s/ && kubectl -n heart-mlops rollout status deploy/heart-api
minikube tunnel                                     # assigns the external IP
curl -X POST http://<EXTERNAL-IP>/predict -H 'Content-Type: application/json' -d
@sample_request.json

# kind with NGINX Ingress (same as CI)
kind create cluster --name heart --config k8s/kind/kind-config.yaml
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-
v1.11.2/deploy/static/provider/kind/deploy.yaml
kind load docker-image heart-api:1.0.0 --name heart && kubectl apply -f k8s/
curl -H 'Host: heart-api.local' http://localhost/health
```

3.5 Monitoring stack

```
docker compose -f monitoring/docker-compose.yml up --build -d    # API :8000,
Prometheus :9090, Grafana :3000
bash monitoring/load_test.sh http://localhost:8000 300
```

Running the full pipeline needs no local setup: any push to `main` (or a manual **Run workflow** from the Actions tab) executes every step above on GitHub-hosted runners.

4. Data Acquisition and Exploratory Data Analysis

4.1 Acquisition and cleaning

`heart/download.py` fetches `processed.cleveland.data` from the UCI Machine Learning Repository, with three retries for transient network errors. The raw file has no header and marks missing values with `?`. Cleaning (`heart/data.py`) applies the documented column schema and converts `?` to NaN. It also treats physiologically impossible zero values of resting blood pressure and cholesterol as missing, and binarises the original 0–4 diagnosis into the target (0 = no disease, 1 = disease). Finally it removes duplicates and validates the schema before anything is trained. The result has **303 rows, 13 features and no duplicates**. Only `ca` (4 rows) and `thal` (2 rows) contain missing values.

Missing values are intentionally **not** filled during cleaning. Imputation lives inside the model pipeline (median for numeric, most-frequent for categorical), so the identical logic is applied to API requests. A request can therefore omit `ca` or `thal` and still be scored.

4.2 Findings

The classes are reasonably balanced: 164 patients (54.1%) without disease and 139 (45.9%) with disease. Plain accuracy is therefore meaningful, but stratified splits are still used throughout. The correlation analysis (Figure 2, right) and per-category disease rates (Figure 4) identify the strongest signals:

- **thal** (thallium stress test) has the strongest correlation with the target (0.53). Reversible defects (7) show a 76% disease rate versus 22% for normal results.
- **ca** (vessels coloured by fluoroscopy, 0.46) has a nearly monotonic effect: the disease rate is 26% with no coloured vessels and 82–85% with two or three.
- **exang** (exercise-induced angina, 0.43) has a 77% disease rate when present versus 31% when absent. **oldpeak** (ST depression, 0.43) behaves similarly.
- **cp = 4** (asymptomatic chest pain) carries a 73% disease rate, much higher than the typical-angina categories. Asymptomatic presentation is a known risk marker.
- **thalach** (maximum heart rate) is negatively correlated (−0.42), so lower exercise capacity indicates higher risk.
- **chol** (0.09) and **fbs** (0.03) are only weakly related to the target on their own. They are kept because the regularised models can down-weight them.

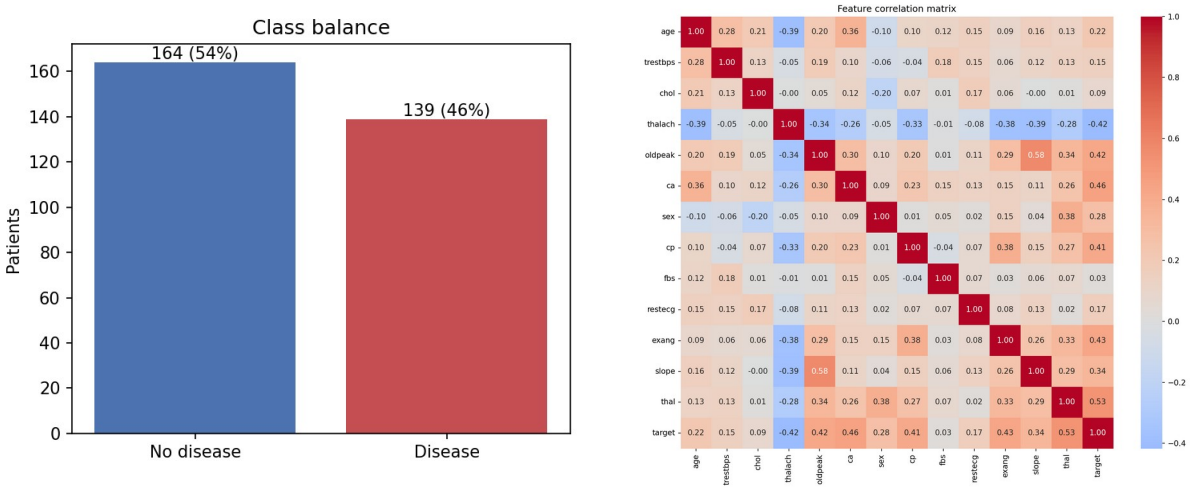


Figure 2. *Class balance (left) and feature correlation matrix (right)*

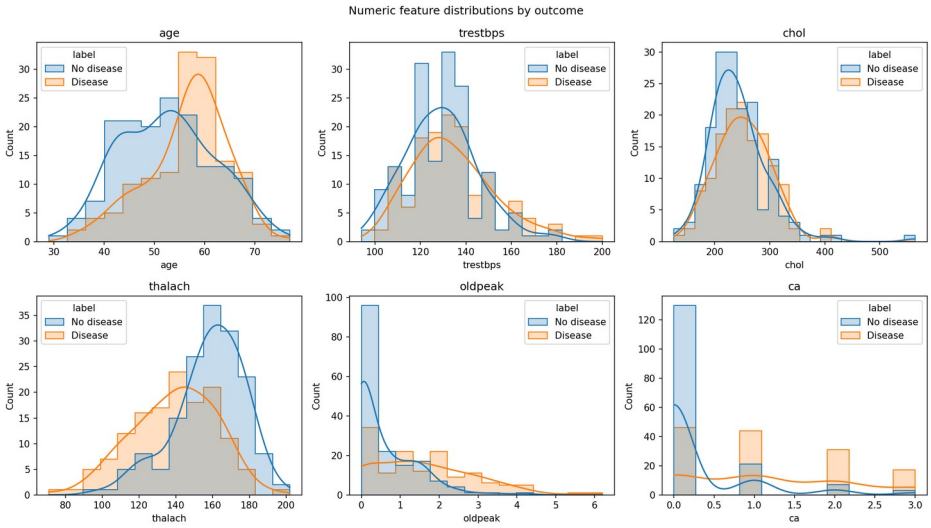


Figure 3. *Distributions of numeric features split by outcome*

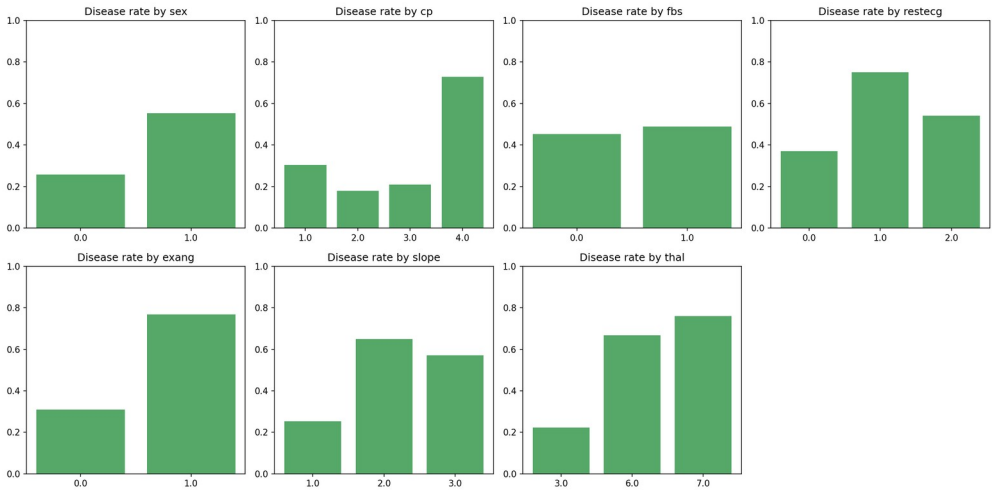


Figure 4. *Disease rate by category for each categorical feature*

The outlier box plots (`reports/figures/05_outliers.png`) show a few extreme cholesterol values (up to 564 mg/dl) and ST depressions up to 6.2. These are clinically plausible, so they were retained. Standard scaling limits their influence on the linear model.

5. Feature Engineering and Model Development

5.1 Feature pipeline

`heart/features.py` defines a `ColumnTransformer` with two branches. **Numeric** features (age, trestbps, chol, thalach, oldpeak, ca) are median-imputed and standardised. **Categorical** features (sex, cp, fbs, restecg, exang, slope, thal) are mode-imputed and one-hot encoded with `handle_unknown="ignore"`, so an unseen category at inference time cannot crash the service. `ca` is kept numeric because it is an ordinal count with a monotonic relationship to risk.

5.2 Candidate models and tuning

Two model families with different inductive biases were compared. Data was split 80/20 (242 training, 61 hold-out) with stratification and a fixed seed of 42. Each candidate was tuned with `GridSearchCV` on the training set using 5-fold stratified cross-validation, optimising ROC-AUC. ROC-AUC was chosen because it is threshold-independent and reflects ranking quality, which matters for risk scoring. The tuned pipeline was then re-evaluated with cross-validation on accuracy, precision, recall, F1 and ROC-AUC, and scored once on the untouched hold-out set.

Table 3. *Hyper-parameter search*

Model	Search space	Best configuration
Logistic Regression (liblinear)	C ∈ {0.01, 0.1, 1, 10}; penalty ∈ {L1, L2}; class_weight ∈ {none, balanced} — 16 fits × 5 folds	C = 0.1, L2, no class weighting
Random Forest	n_estimators ∈ {200, 400}; max_depth ∈ {none, 4, 8}; min_samples_leaf ∈ {1, 3, 5} — 18 fits × 5 folds	200 trees, max_depth = 4, min_samples_leaf = 5

Table 4. *Cross-validated and hold-out performance (real UCI data, CI run #1)*

Metric	Logistic Regression	Random Forest
CV ROC-AUC (mean ± sd)	**0.912 ± 0.017**	0.900 ± 0.032
CV accuracy	0.847 ± 0.029	0.810 ± 0.037
CV precision	0.869 ± 0.066	0.814 ± 0.050
CV recall	0.792 ± 0.048	0.764 ± 0.089
CV F1	0.826 ± 0.030	0.784 ± 0.047

Metric	Logistic Regression	Random Forest
Hold-out ROC-AUC	**0.962**	0.955
Hold-out accuracy / precision / recall / F1	0.885 / 0.839 / 0.929 / 0.881	0.885 / 0.839 / 0.929 / 0.881

5.3 Model selection

The champion is selected automatically on **mean cross-validated ROC-AUC**, not on the single 61-row hold-out set, which is too small to separate close models reliably. Logistic Regression wins on every cross-validated metric and has roughly half the fold-to-fold variance of the Random Forest. It is also directly interpretable through its coefficients (`reports/model_plots/`). The tuned forest converged to shallow, heavily regularised trees (depth 4, leaf size 5), which suggests the dataset is too small for a high-variance learner to show an advantage.

On the hold-out set the champion reaches 0.962 ROC-AUC with **recall 0.929**: it misses only 2 of 28 disease cases (Figure 5, right). In a screening setting false negatives are the costlier error, so this is the most important number. The decision threshold is the default 0.5. The API also returns the raw probability, so a client can apply a more conservative threshold.

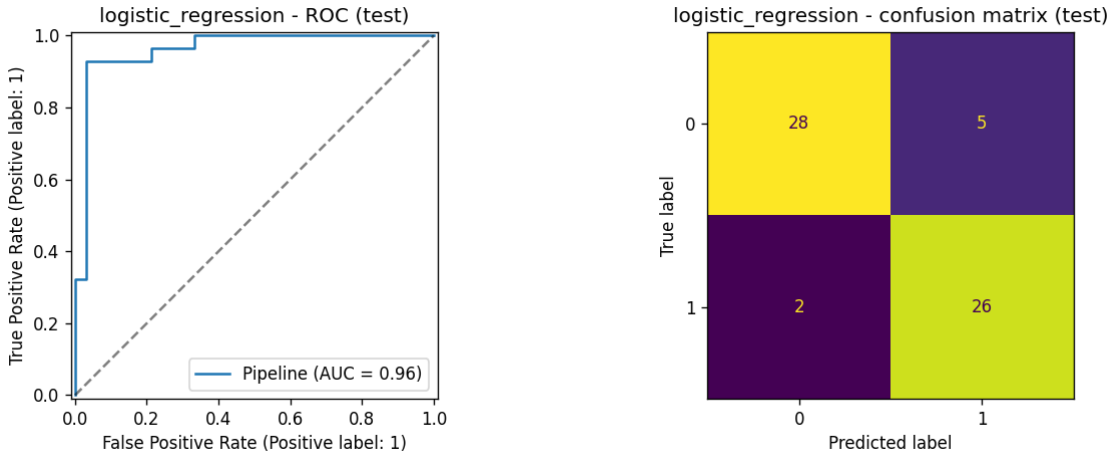


Figure 5. *Champion model on the hold-out set: ROC curve (left) and confusion matrix (right)*

6. Experiment Tracking with MLflow

Every training run is tracked in MLflow (experiment `heart-disease`). For each candidate the pipeline logs the following:

- **Parameters:** model type, best hyper-parameters, CV folds, seed, train/test sizes and grid size.
- **Metrics:** mean and standard deviation of all five CV metrics, plus the five hold-out metrics.
- **Artifacts:** the full grid-search results (CSV), confusion matrix, ROC curve and feature-importance plots.
- **Model:** the full sklearn pipeline with an inferred input/output signature and an input example, so it can be served by MLflow directly.

After both runs finish, the winning run is tagged `champion=true`, and its run ID is written into `models/metadata.json`. This links any deployed model version back to its experiment. In CI the `mlruns/` directory is uploaded as a workflow artifact, and the MLflow UI is captured automatically (Figure 6).

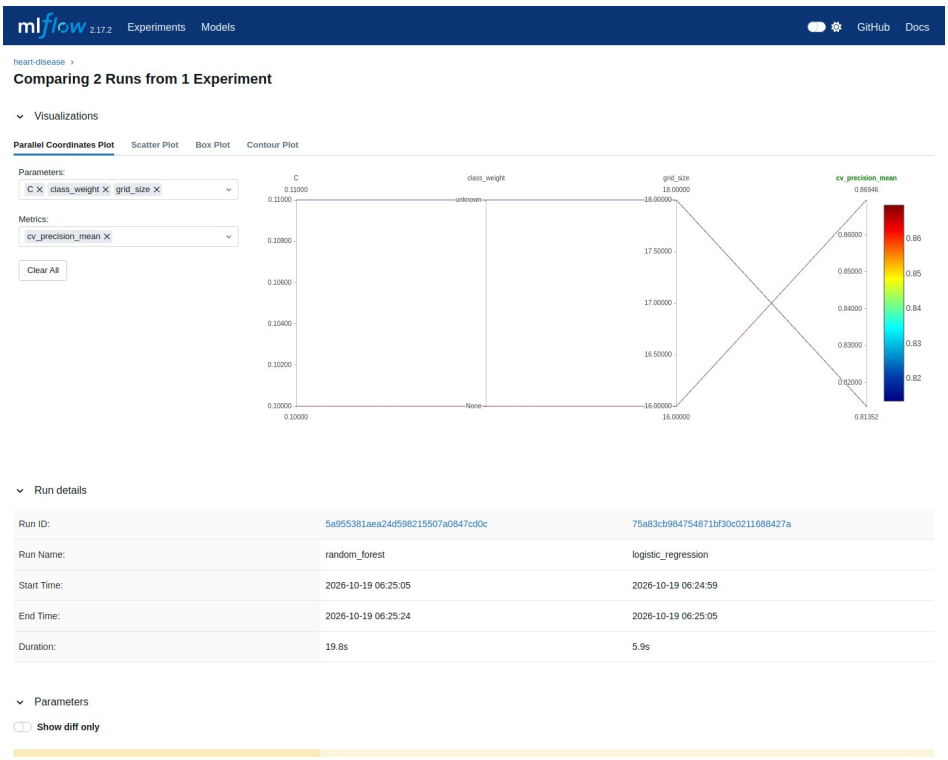


Figure 6. *MLflow run comparison: parameters and metrics side by side*

7. Model Packaging and Reproducibility

- **Single artifact:** `models/model.joblib` contains preprocessing and classifier together. `metadata.json` records the model name, version timestamp, MLflow run ID, scikit-learn version, feature order, best parameters and all metrics.
- **Pinned environments:** every package is pinned to an exact version. The serving image uses a smaller `requirements-serve.txt` with the same pins as training, so the pickled pipeline is always loaded by the scikit-learn version that produced it (1.5.2).
- **Determinism:** all splits, CV folds and models use seed 42. Re-running `heart.train` on the committed `data/processed/heart_clean.csv` on a separate machine reproduced CI run #1 exactly: CV ROC-AUC 0.9117 and hold-out ROC-AUC 0.9621 for the champion.
- **Clean-setup guarantee:** each CI job starts from a fresh runner and installs only from the requirements files, which proves that every script runs from a clean setup.
- **Inference script:** `python -m heart.predict sample_request.json` provides offline batch scoring with the same artifact.

8. CI/CD Pipeline and Automated Testing

8.1 Unit tests

The `test/` folder contains 20 Pytest tests that run against synthetic data in the UCI schema, so the test stage never depends on the network:

- **test_data.py (7 tests):** ? parsing; binary target; zero-cholesterol handling; duplicate removal; column order; schema and target validation errors.
- **test_features.py (3 tests):** no NaNs after preprocessing; numeric scaling; unseen categories handled.
- **test_model.py (5 tests):** at least two candidates exist; model and metadata are exported; probabilities are valid; the model learns signal (CV ROC-AUC > 0.6); runs are logged to MLflow with exactly one champion.
- **test_api.py (5 tests):** /health works; /predict returns a prediction, confidence and request ID; optional fields can be omitted; invalid input is rejected with HTTP 422; Prometheus metrics are exposed.

8.2 Pipeline design

Table 5. *GitHub Actions jobs (total run time 8 min 34 s)*

Job	What it does	Run #1 time
lint	ruff lint and format check; fails the pipeline on any violation	9 s
test	Pytest with coverage; JUnit XML and coverage reports uploaded as artifacts	1 m 40 s
train	Downloads real UCI data, runs EDA and training; publishes the model, MLflow runs and plots; writes the champion metrics to the job summary	1 m 13 s
docker	Builds the image, runs it, asserts /health, a valid /predict and a 422 for bad input; pushes to GHCR on main	53 s
deploy-k8s	Creates a kind cluster with NGINX Ingress, deploys the manifests, waits for rollout and calls the API through the Ingress	2 m 10 s
monitoring	Starts MLflow UI, API, Prometheus and Grafana; generates traffic; captures UI screenshots	3 m 49 s
publish-evidence	Renders logs and commits the cleaned dataset, figures and screenshots back to the repository	35 s

Jobs are chained with `needs`, so a lint error or failing test stops the pipeline before training or deployment. Every shell step runs with `set -o pipefail`, which means a failure is never masked by `tee`. On failure, dedicated steps dump pod descriptions and service logs, which keeps the logs clear. Each run uploads ten artifacts, including test reports, the model, MLflow runs, EDA plots and evidence logs.

Issue found and fixed. A manual re-run of an older run failed in the publish job. The job had checked out the old commit, while `main` already contained evidence from a later run, so the push hit merge conflicts on the same screenshot files. The fix checks out the latest branch head (`ref: ${{ github.ref_name }}`) and resolves any concurrent update in favour of the newest evidence (`git pull --rebase -X theirs`). Run #4 then passed all seven jobs in 7 min 59 s with identical model metrics, which confirms both the fix and the reproducibility of training.

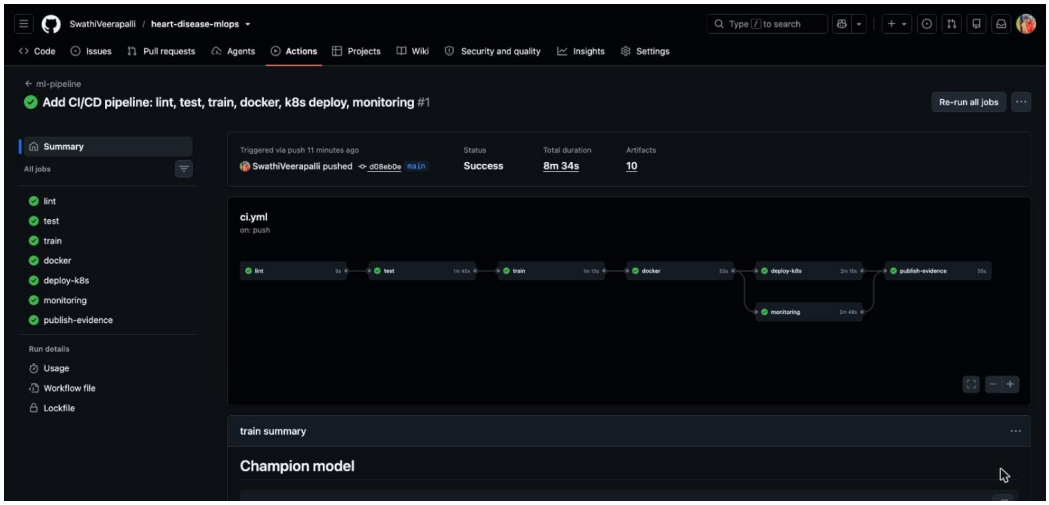


Figure 7. *GitHub Actions run #1: all seven jobs succeeded*

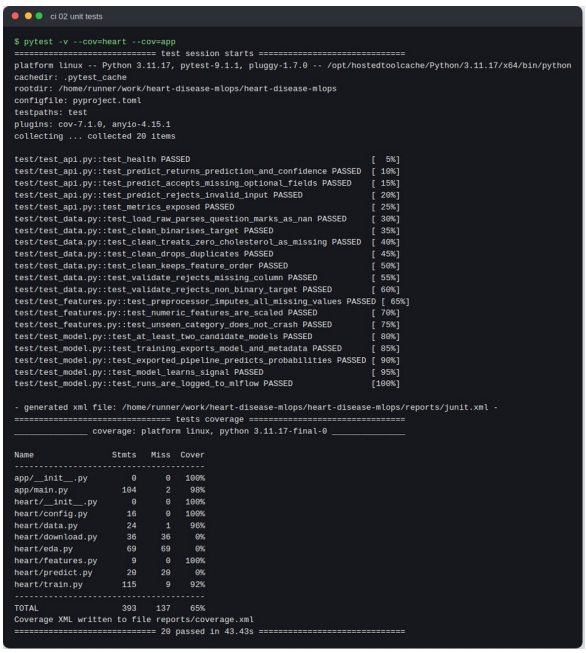


Figure 8. *Unit-test stage output in CI: 20 tests passed*



Figure 9. *Training stage on the real dataset: download, cleaning, EDA and model selection*

9. Model Containerisation and API

The API (`app/main.py`) is built with FastAPI. Requests are validated by a Pydantic schema with clinical range checks: for example, age must be 1–120, cp must be 1–4 and thal must be 3, 6 or 7. Invalid input is rejected with HTTP 422 before it reaches the model. The model loads once at startup.

Table 6. *API contract*

Endpoint	Purpose
POST /predict	JSON patient record → prediction (0/1), label, confidence, probability_disease, model_version, request_id
GET /health	Liveness/readiness probe; 503 if the model is not loaded
GET /model-info	Model metadata (name, version, metrics, MLflow run ID)
GET /metrics	Prometheus exposition format
GET /docs	Interactive Swagger documentation with a ready-to-run example request

The Dockerfile is based on `python:3.11-slim` and installs only the serving requirements. It runs as a non-root user (UID 10001) and defines a `HEALTHCHECK`. It uses a single Uvicorn worker per container, so the Prometheus counters stay consistent, and scaling is handled by Kubernetes replicas instead. The CI docker job proves the

container builds and serves correctly in isolation (Figure 10). The interactive Swagger UI is captured in `screenshots/api_01_swagger_docs.png`.

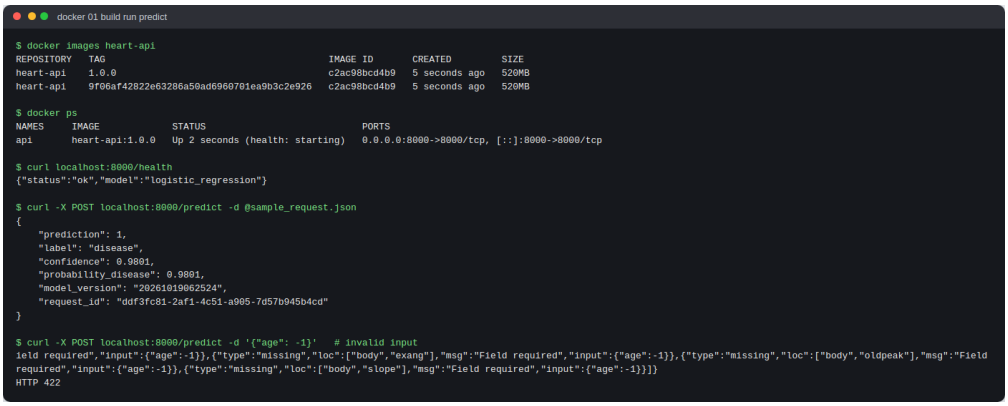


Figure 10. *Container build-and-run proof from CI: image, running container, health check, prediction and input validation*

10. Production Deployment on Kubernetes

The manifests in `k8s/` define a dedicated namespace and a **Deployment** with 2 replicas and rolling updates (maxUnavailable 0). The pods have readiness and liveness probes on /health and CPU and memory requests and limits, and they run with a hardened security context (non-root, read-only root filesystem, no privilege escalation). Pod annotations enable Prometheus scraping. Traffic reaches the pods through a **LoadBalancer Service** and an **NGINX Ingress** (`heart-api.local`). A **HorizontalPodAutoscaler** scales between 2 and 5 replicas at 70% CPU.

In CI the deployment is verified on an ephemeral kind cluster with the NGINX Ingress controller. Both pods become ready, and real requests are routed through the Ingress (Figures 11–12). The same manifests deploy to Minikube (Section 3.4) or a managed cluster such as EKS, GKE or AKS. For a managed cluster, point the Deployment image at the published GHCR image `ghcr.io/swathiveerapalli/heart-disease-mlops/heart-api:latest`.

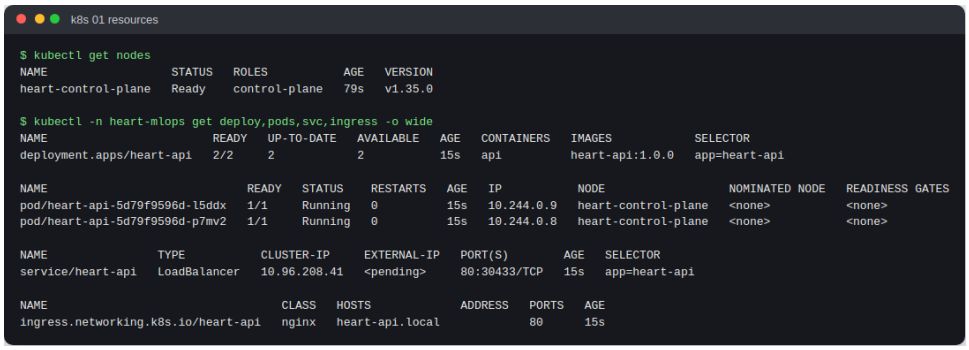


Figure 11. *Kubernetes resources: node, deployment (2/2 ready), pods, LoadBalancer service and ingress*

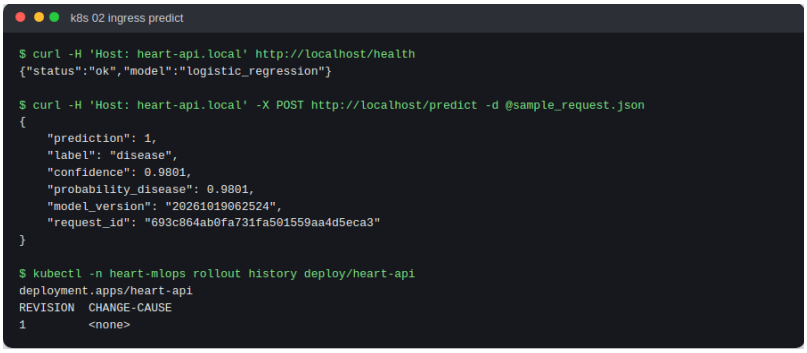


Figure 12. *Health check and prediction through the NGINX Ingress*

11. Monitoring and Logging

Logging. A middleware writes one structured JSON line per request, containing the request ID, method, path, status, latency and client. A second line records each prediction with its confidence. The request ID is also returned in the `x-request-id` header, so a client-side problem can be matched to a server log line. Because the logs go to stdout, they are collected by `docker logs` and `kubectl logs` and can be shipped by any standard log agent (Figure 13).

Metrics. The service exposes `api_requests_total` (by method, path and status), the `api_request_latency_seconds` histogram, `model_predictions_total` by predicted class, a `model_prediction_confidence` histogram and a `model_loaded` gauge. Prometheus scrapes them every 10 seconds. Grafana is provisioned automatically with a dashboard showing request rate, p95 latency, predictions by class, error rate, median confidence and model status.

In the monitoring CI job, about 500 synthetic requests (5% deliberately invalid) were sent to the API. The dashboard shows a p95 latency of about 5 ms after warm-up and an error-rate spike matching the injected bad requests. It also shows a stable split between predicted classes, which is the basis for detecting prediction drift (Figure 14). Prometheus target health is in `screenshots/monitoring_01_prometheus_targets.png`.

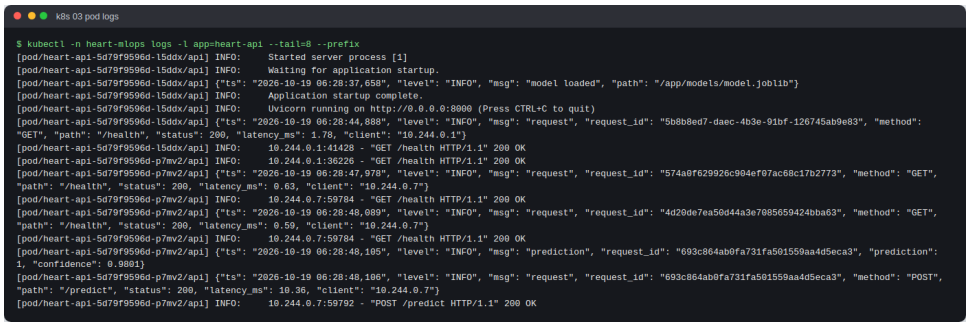


Figure 13. *Structured JSON request and prediction logs from the Kubernetes pods*



Figure 14. *Grafana dashboard generated from live traffic in the CI monitoring job*

12. Deliverables and Access

Table 7. *Deliverables checklist*

Required deliverable	Location
Code, Dockerfile, requirements	`heart/`, `app/`, `Dockerfile`, `requirements*.txt`

Required deliverable	Location
Cleaned dataset + download script	`data/processed/heart_clean.csv`, `heart/download.py`
Scripts for EDA, training, inference	`heart/eda.py`, `heart/train.py`, `heart/predict.py`
test/ folder with unit tests	`test/` (20 tests)
GitHub Actions workflow	`.github/workflows/ci.yml`
Deployment manifests	`k8s/` (+ `k8s/kind/kind-config.yaml`)
Screenshot folder	`screenshots/` (CI, Docker, Kubernetes, MLflow, monitoring)
Final report	This document
API access	Local or Kubernetes deployment per Section 3; container image on GHCR

Repository: https://github.com/SwathiVeerapalli/heart-disease-mlops

Pipeline run: https://github.com/SwathiVeerapalli/heart-disease-mlops/actions/runs/39206495186

13. Conclusion

The project delivers a reproducible, automated path from the raw UCI dataset to a monitored prediction service. A regularised Logistic Regression was selected over a Random Forest on cross-validated ROC-AUC (0.912 vs 0.900). It generalises to the hold-out set with 0.962 ROC-AUC and 93% recall. Every stage is enforced by CI, which includes quality gates, a container smoke test, a real Kubernetes rollout behind an Ingress, and live monitoring.

Natural next steps would be:

- Calibrate probabilities and choose the decision threshold from a clinical cost matrix.
- Add data-drift checks on incoming feature distributions.
- Promote models through the MLflow Model Registry with a manual approval gate before production.
- Deploy to a managed cluster with the AWS Load Balancer Controller or equivalent for a persistent public endpoint.